

Q 值与动作概率的计算过程—讲解说明

DQN 的 Q 值计算

网络结构

```

输入状态 (3×12×12)          3 个通道：蛇身、蛇头、食物位置
↓
Conv2D (3→32, stride=2)      12×12 → 6×6, 提取空间特征
↓
FC (1152 → 256)              全连接层, 特征融合
↓
FC (256 → 4)                  输出 4 个神经元
↓
[Q_up, Q_down, Q_left, Q_right]
↑ 输出层无激活函数, 值为任意实数

```

计算过程

1. 游戏画面编码为 3 通道图像 (蛇身、蛇头、食物各一通道), 形状 (3, 12, 12)
2. 经过一层 stride=2 卷积, 空间尺寸压缩到 (32, 6, 6), 保留位置信息
3. 展平后经过两层全连接层 (256 隐藏层 $\rightarrow$ 4 输出层)
4. 输出层没有激活函数, 直接输出 4 个原始 Q 值

Q 值的含义

- 每个 Q 值 = “在当前状态执行该动作，未来能获得的累积奖励”的预期
- 值域：任意实数（负值表示预期会导致失败）
- 训练目标：让 Q 值逼近真实的最优价值函数 $Q^*(s, a)$

PPO 的动作概率计算

网络结构 (Actor-Critic 共享 CNN)

```

输入状态 (3×12×12)
    ↓
Conv2D × 3 (3→32→64→64)    12×12 → 6×6 → 3×3, 三层卷积 + BN + ReLU
    ↓
共享 FC (576 → 256)
    ↓
Actor FC (256 → 4)    Softmax    [P_up, P_down, P_left, P_right]
                                   ↑ 概率分布, 和为 1
Critic FC (256 → 1)    V(s)
                                   ↑ 状态价值标量

```

计算过程

1. 同样输入 3 通道画面，经过**三层卷积**（更深，带 BatchNorm）提取特征
2. 共享全连接层后分为两个分支：
 - **Actor 头**: $FC(256 \rightarrow 4) \rightarrow \text{Softmax}$ ，输出四个动作的概率分布
 - **Critic 头**: $FC(256 \rightarrow 1)$ ，输出当前状态的标量价值 $V(s)$
3. Actor 输出层使用 **Softmax 激活**，确保输出为正且总和为 1

概率的含义

- 每个概率值 = “策略网络推荐该动作的置信度”
- 值域: $[0, 1]$, $\sum P(action) = 1$
- 训练时按概率采样动作（随机策略），评估时取最大概率的动作（确定性策略）

对比总结

方面	DQN 的 Q 值	PPO 的动作概率
网络结构	CNN $\rightarrow$ FC $\rightarrow$ 单头输出	共享 CNN $\rightarrow$ Actor + Critic 双头
输出激活	无 （线性层直接输出）	Softmax 归一化
数值含义	预期累积奖励 $Q(s, a)$	策略置信度 $\pi(a s)$
数值范围	任意实数	$[0, 1]$ ，总和为 1
决策方式	选 Q 值最大的动作（贪心）	按概率采样（训练） / 取最大（评估）
更新方式	Off-policy，从经验池采样更新	On-policy，用当前策略收集的轨迹更新

可视化映射

两者在可视化时采用相同的处理方式：

1. 取出蛇头四个方向的数值（上、下、左、右）
2. 对四个数值做 min-max 归一化到 $[0, 1]$
3. 映射为色块颜色：红色（高） $\rightarrow$ 蓝色（低），半透明叠加在对应方向的格子上

听众无需理解底层数学区别，通过颜色即可直观感知 AI 的”决策意图”。